# Inputs on an Embedded Linux Device

On your Embedded Linux device, when there's no windowing system present, the mouse, keyboard, and touch input are read directly via evdev or using helper libraries such as libinput or tslib. However, this behavior requires that device nodes /dev/input/event* are readable by the user. eglfs and linuxfb have all the input handling code compiled-in.

## Use libinput

libinput is a library to handle input devices that offers an alternative to the Qt's own evdev input support. To enable using libinput, when you configure and build Qt, make sure that the development files for libudev and libinput are available. If you require keyboard support, then xkbcommon is also necessary. With eglfs and linuxfb, no further actions are necessary as these plugins use libinput by default. If libinput support is not available or the QT_QPA_EGLFS_NO_LIBINPUT environment variable is set, then Qt's own evdev handlers are used instead.

The following environment variables are available when input events are received via libinput:

| Variable | Description |
|---|---|
| QT_QPA_LIBINPUT_TOUCH_MATRIX | Specifies the first two rows of the 3x3 transformation matrix to apply. The values are expected to be separated by spaces. See the libinput documentation for details. |

## Input on eglfs and linuxfb without libinput

Parameters like the device node name can be set in the QT_QPA_EVDEV_MOUSE_PARAMETERS, QT_QPA_EVDEV_KEYBOARD_PARAMETERS and QT_QPA_EVDEV_TOUCHSCREEN_PARAMETERS environment variables; separate your entries with colons. These parameters are an alternative to passing the settings in the -plugin command-line argument, and with some backends they are essential. But eglfs and linuxfb use built-in input handlers so there's no separate -plugin argument in use.

Additionally, the built-in input handlers can be disabled by setting QT_QPA_EGLFS_DISABLE_INPUT (for eglfs) or QT_QPA_FB_DISABLE_INPUT (for linuxfb) to 1.

## Mouse

The mouse cursor shows up whenever QT_QPA_EGLFS_HIDECURSOR (for eglfs) or QT_QPA_FB_HIDECURSOR (for linuxfb) isn't set and Qt's libudev-based device discovery reports that at least one mouse is available. When libudev support is not present, the mouse cursor is always displayed; unless it's explicitly disabled via the environment variable.

If Qt was configured with libudev support, connecting or disconnecting an input device while the application is running (hot plugging) is supported. Then libudev development headers are present in the sysroot at configure time.

The evdev mouse handler supports the following extra parameters:

| Parameter | Description |
|---|---|
| /dev/input/... | Specifies the name of the input device. If unspecified, Qt looks for a suitable device either via libudev or by traversing the available nodes. |
| nocompress | By default, input events that don't lead to changing the position compared to the last Qt mouse event are compressed. A new Qt mouse event is sent only after a change in the position or button state. To disable this behavior, set the nocompress parameter. |
| dejitter | Specifies a jitter limit; disabled by default. |
| grab | When set to 1, Qt grabs the device for exclusive use. |
| abs | Some touchscreens report absolute coordinates and can't be differentiated from touch pads. In this case, pass abs to indicate that the device is using absolute events. |

## Keyboard

The evdev keyboard handler supports the following extra parameters:

| Parameter | Description |
|---|---|
| /dev/input/... | Specifies the name of the input device. If unspecified, Qt looks for a suitable device either via libudev or by traversing the available nodes. |
| grab | Enables grabbing the input device. |
| disable-zap | Disables the zap key chord. See the note on **Ctrl+Alt+Backspace** below. |
| keymap | Specifies the name of a custom keyboard map file. |
| enable-compose | Enables compositing. |
| repeat-delay | Sets a custom key repeat delay. |
| repeat-rate | Sets a custom key repeat rate. |

On Embedded Linux systems that don't have their terminal sessions disabled, the behavior on a key press can be confusing, as the input event is processed by the Qt application and the tty. To overcome this, the following options are available:

- An alternative approach is to use the `evdev` keyboard handler's `grab` parameter by passing `grab=1` in `QT_QPA_EVDEV_KEYBOARD_PARAMETERS`. This results in trying to get a grab on the input device. If the `grab` is successful, no other components in the system receive events from it, as long as the Qt application is running. This approach is more suitable for applications that start remotely as it doesn't need access to the tty device.
- Finally, for many specialized Embedded Linux images it doesn't make sense to have the standard terminal sessions enabled in the first place. For more details on how to disable these terminal sessions, refer to your build environment's Documentation. For example, when generating images using the **Yocto Project**, unsetting `SYSVINIT_ENABLED_GETTYS` results in having no `getty` process running. This means, there's no input on any of the virtual terminals.

If the default built-in keymap is not sufficient, you can specify a different one via the `keymap` parameter.

> **Note:** As with other configuration files for `eglfs`, these legacy keymap files are considered trusted content that is fully controlled and managed by the device or platform creator. They are not expected to be exposed to the end-users of the device, and are not meant to allow the loading of user-provided keymap files.

> **Warning:** With the built-in keymap, the zap chord (**Ctrl+Alt+Backspace**) quits the application. The chord is consumed by the keyboard handler, so the application is never notified about it and cannot veto it. On a device where any keyboard-like input device can therefore shut down the user interface, pass `disable-zap` in `QT_QPA_EVDEV_KEYBOARD_PARAMETERS`.

> **Note:** Console switching (**Ctrl+Alt+Fx**) and the reboot chord are recognized by the keyboard handler but not acted upon, and are therefore effectively ignored.

To generate a custom keymap, use the `kmap2qmap` utility, that can be found in the `qttools` module. The source files have to be in standard Linux `kmap` format, which is understood by the kernel's `loadkeys` command. `qmap` files can be generated in one of the following ways:

- The **Linux Console Tools (LCT)** project.
- **X.org** X11 keymaps can be converted to the `kmap` format with the `ckbcomp` utility.
- As `kmap` files are plain-text files, they can also be hand crafted.

`kmap2qmap` is a command line program, that needs at least 2 files as parameters. The last parameter is the generated `.qmap` file, while all the others are parsed as input `.kmap` files. For example:

```
kmap2qmap i386/qwertz/de-latin1-nodeadkeys.kmap include/compose.latin1.inc de-latin1-nodeadkeys.qmap
```

> **Note:** `kmap2qmap` doesn't support all the (pseudo) symbols that the Linux kernel supports. Consequently, when you convert a standard keymap, there'll be a number of warnings regarding `Show_Registers`, `Hex_A`, and so on; these messages can be ignored.

## Touch

While it's not necessary for modern touchscreens, some older resistance touchscreens that only support single-touch may require that you fall back to using `tslib`, instead of relying on the Linux multi-touch protocol and the event devices.

To enable `tslib` support, set the `QT_QPA_EGLFS_TSLIB` (for `eglfs`) or `QT_QPA_FB_TSLIB` (for `linuxfb`) environment variable to 1. To change the device, set the `TSLIB_TSDEVICE` environment variable or pass the device name on the command-line. Note that the `tslib` input handler generates mouse events and supports single touch only, as opposed to `evdevtouch` which generates true multi-touch **QTouchEvent** events too.

> **Note:** Device selection and filtering are handed over to `tslib` entirely. This means that tslib's own configuration file determines which filter modules are loaded into the application's process; Qt neither selects nor enumerates them. Prefer `libinput` for new devices.

The `evdev` touch handler supports the following extra parameters:

| Parameter | Description |
|---|---|
| `/dev/input/...` | Specifies the name of the input device. If unspecified, Qt looks for a suitable device either via `libudev` or by traversing the available nodes. |
| `rotate` | On some touch screens the coordinates must be rotated by setting `rotate` to 90, 180, or 270. |
| `invertx` and `inverty` | Specifies the parameters to invert the X or Y coordinates in the input events. |

For example, if you pass the following values to `QT_QPA_EVDEV_TOUCHSCREEN_PARAMETERS` before launching applications, you'd have an explicitly specified touch device with the coordinates flipped. This is useful when the orientation of the actual screen and the touch screen don't match.

```
export QT_QPA_EVDEV_TOUCHSCREEN_PARAMETERS=/dev/input/event5:rotate=180
```

## Pen-based Tablets

The plugin can take a device node parameter, such as `QT_QPA_GENERIC_PLUGINS=evdevtablet:/dev/event1`, if Qt's automatic device discovery (based either on `libudev` or traversing `/dev/input/event*`) isn't functional or is misbehaving.

### Debug Input Devices

It's possible to print some information to the debug output by enabling the `qt.qpa.input` logging rule, for example by setting the `QT_LOGGING_RULES` environment variable to `qt.qpa.input=true`. This is useful for detecting which device is being used, or for troubleshooting device discovery issues.

For more detailed logging of input subsystems (mouse, keyboard, touch), enable all input-related logging categories with:

```
export QT_LOGGING_RULES="qt.qpa.input.*=true;qt.qpa.events=true"
```

### Use Custom Mouse Cursor Images

`eglfs` comes with its own set of 32x32-sized mouse cursor images. If these are insufficient, you can provide a custom cursor atlas by setting the `QT_QPA_EGLFS_CURSOR` environment variable to the name of a JSON file. This file can also be embedded into the application via The Qt Resource System.

For example, an embedded cursor atlas with 8 cursor images per row can be specified as follows:

```
{
  "image": ":/cursor-atlas.png",
  "cursorsPerRow": 8,
  "hotSpots": [
      [7, 2],
      [12, 3],
      [12, 12],
      ...
  ]
}
```

Note that the images are expected to be tightly packed in the atlas; the width and height of the cursors are determined based on the total image size and the `cursorsPerRow` setting. Atlases must also provide an image for all of the supported cursors.

> **Note:** As with other configuration files for `eglfs`, the cursor atlas is considered trusted content that is fully controlled and managed by the device or platform creator. It is not expected to be exposed to the end-users of the device.

The `image` and `hotSpots` keys and a non-zero `cursorsPerRow` are all mandatory, and `hotSpots` must have one entry for every supported cursor shape. A file that is missing any of these is rejected with a warning and the cursor is disabled.

‹ Configure an Embedded Linux Device

Fonts ›

© 2026 The Qt Company Ltd. Documentation contributions included herein are the copyrights of their respective owners. The documentation provided herein is licensed under the terms of the GNU Free Documentation License version 1.3 as published by the Free Software Foundation. Qt and respective logos are trademarks of The Qt Company Ltd. in Finland and/or other countries worldwide. All other trademarks are property of their respective owners.

QtWS26 The Qt Event of the Year: Qt World Summit 2026 Save Your Spot ›

Contact Us
Qt Group
Our Story
Brand
News
Careers
Investors
Qt Products
Software Quality Products
Licensing
License Agreement
Open Source
Plans and pricing
Download
FAQ
Learn Qt
For Learners
For Students and Teachers
Qt Documentation
Qt Forum
Support & Services
Professional Services
Customer Success
Support Services
Partners

Qt Group | DOCUMENTATION

Qt Group includes The Qt Company Oy and its global subsidiaries and affiliates.